

Autonomous Civic Monitoring App

Best-practice workflow, AI pipeline, tech stack and app architecture



ZERO MANUAL CAPTURE

Dashcam + AI automatically detect issues while the vehicle moves.



EDGE-FIRST AI

Only relevant evidence is saved/uploaded; continuous raw video need not leave the vehicle.



ISSUE, NOT PHOTO

Many detections can corroborate one civic issue instead of creating duplicate complaints.

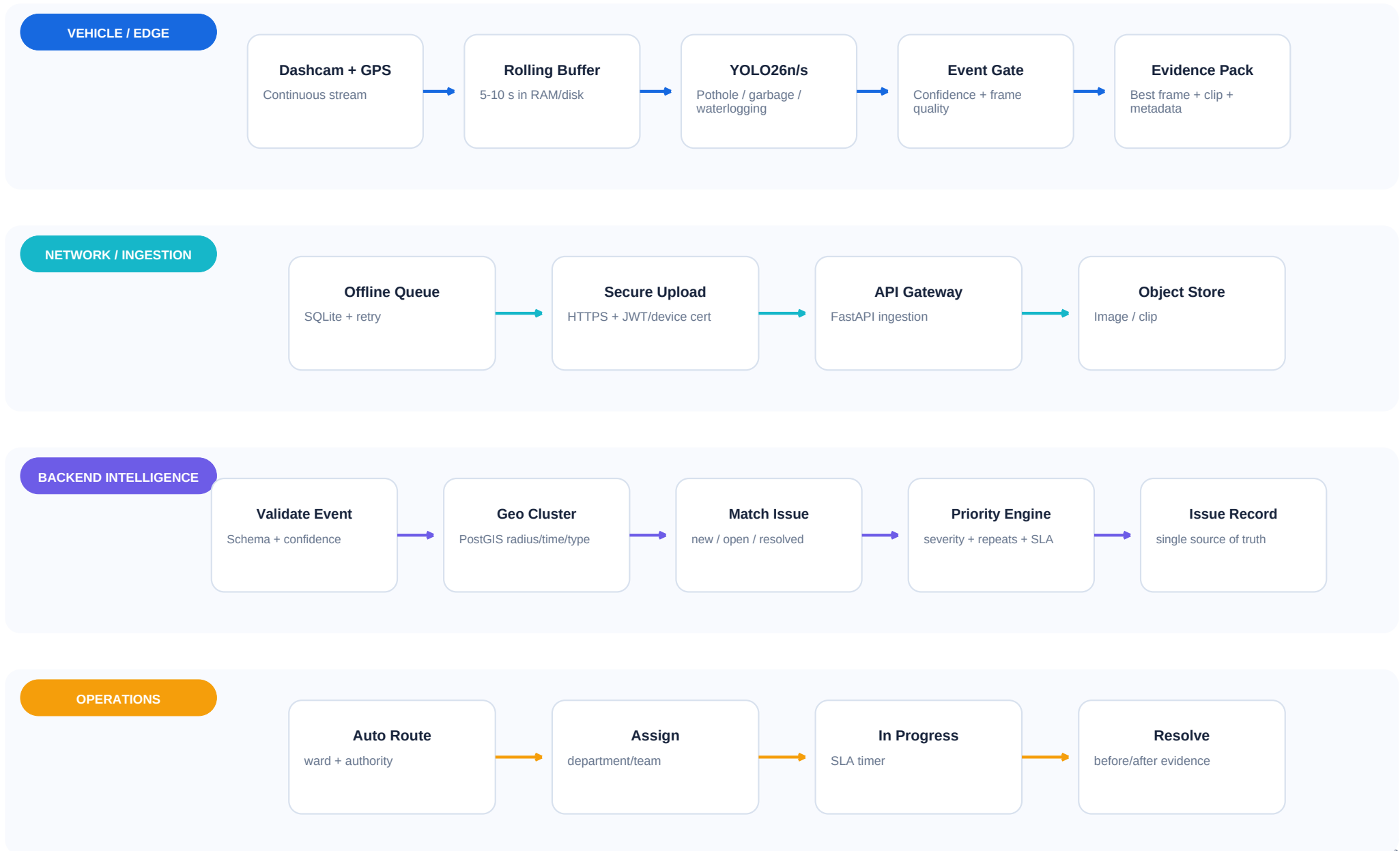


CLOSED LOOP

Detect -> assign -> resolve -> independently re-verify -> close or reopen.

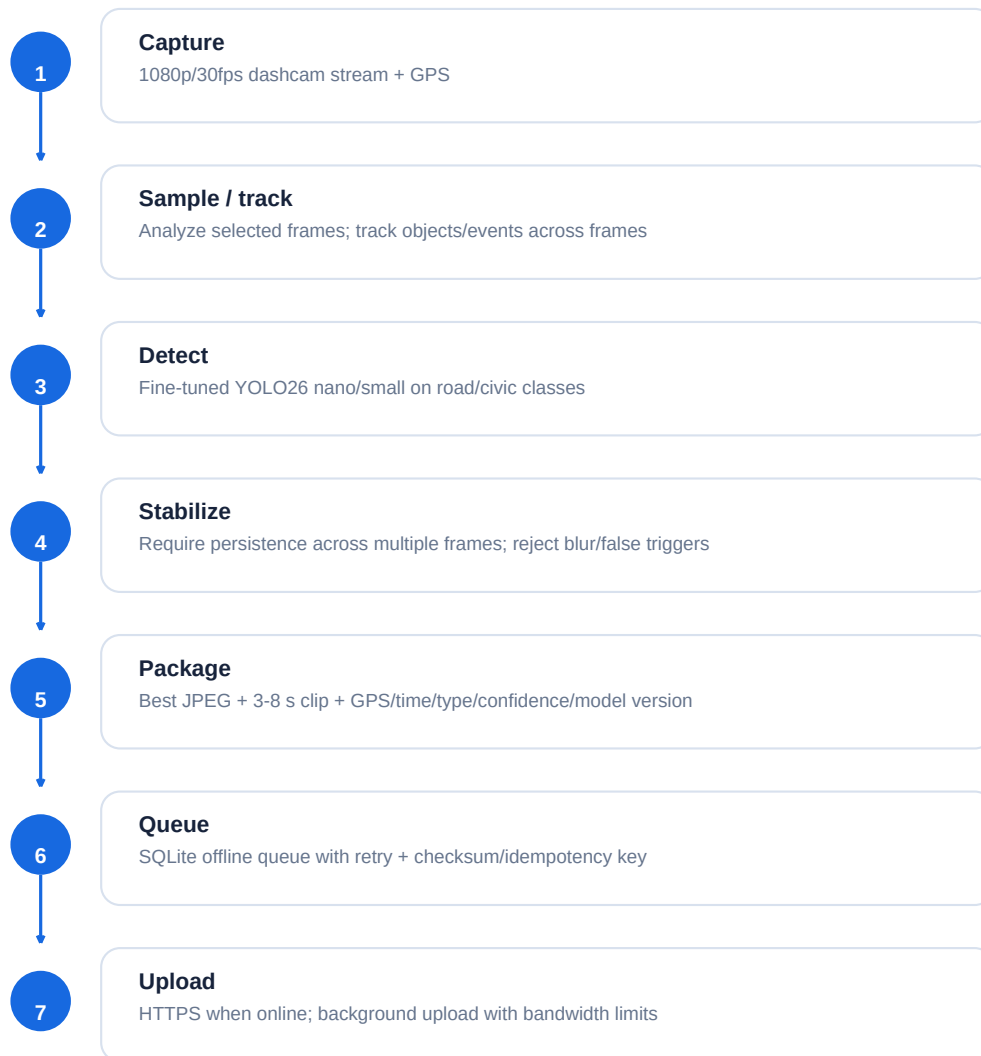
1. End-to-End Workflow

The app is the control layer; the camera/edge unit performs detection automatically.



2. Edge AI Pipeline

Recommended: process video on the vehicle; upload only evidence and metadata.



Recommended edge stack

COMPUTE **NVIDIA Jetson Orin Nano Super (MVP) / Orin NX for heavier multi-camera loads**
67 INT8 TOPS on the Nano Super; 7-25 W configurable power.

VIDEO **GStreamer + OpenCV**
Reliable camera ingestion, decoding, frame selection and clip extraction.

MODEL **Ultralytics YOLO26n / YOLO26s, custom fine-tuned**
Start nano for latency; move to small only if field accuracy needs it.

INFERENCE **TensorRT FP16; INT8 after calibration**
Optimized NVIDIA inference. Validate accuracy before enabling INT8.

LOCAL STATE **SQLite + filesystem/NVMe ring buffer**
Simple, robust offline-first queue and evidence cache.

DEVICE SERVICES **Python/C++ worker + systemd + watchdog**

Privacy rule

Do not continuously upload raw road video. Keep the rolling buffer local and transmit only evidence linked to a detected event, unless policy explicitly requires archival.

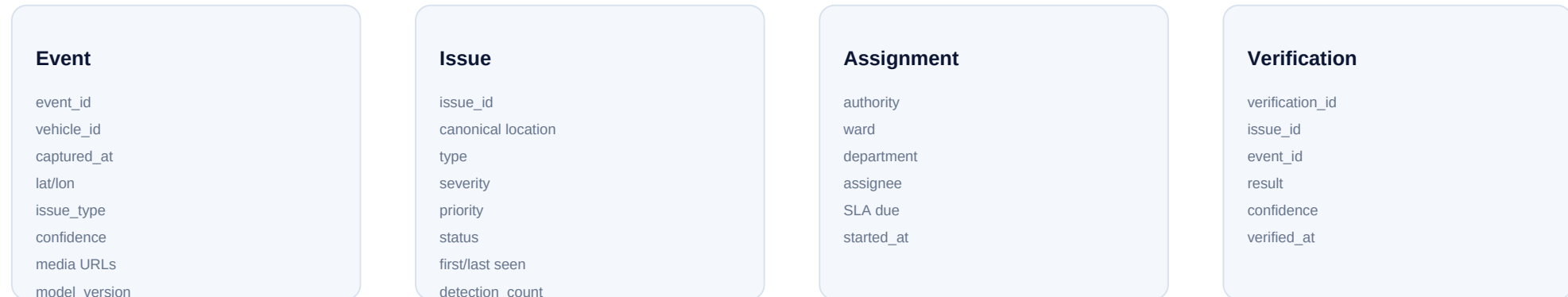
3. Backend Intelligence & Data Model

Make every camera detection an Event; make the real-world problem an Issue.

Backend pipeline



Core data model



Issue lifecycle

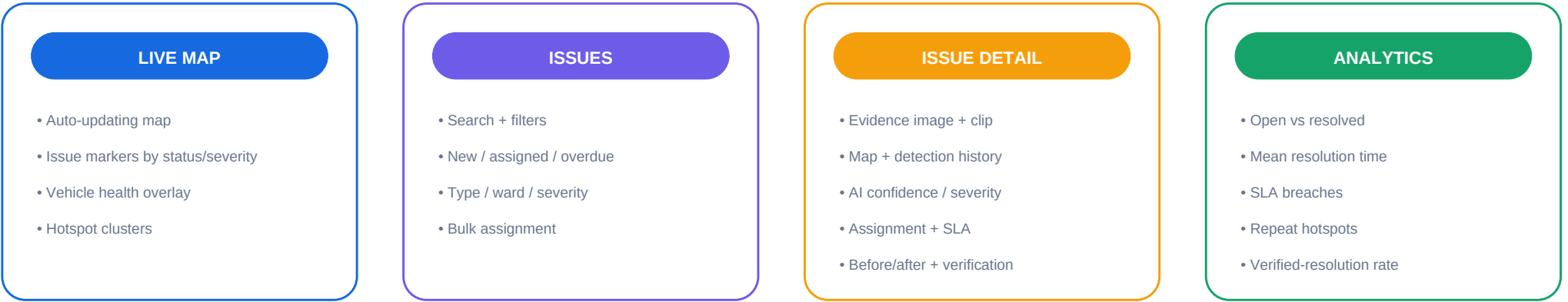


Failure branches: REJECTED / DUPLICATE-REFERENCE / REOPENED / ESCALATED

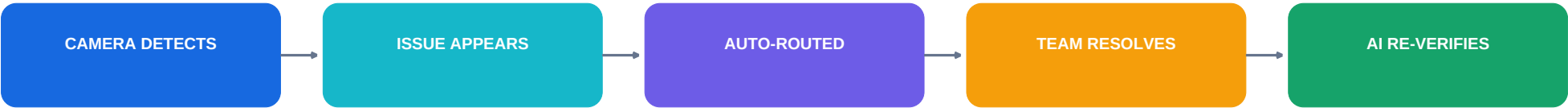
4. App Experience

No 'Take Photo' button. The app receives issues automatically from the camera fleet.

Recommended information architecture



What happens without any manual capture



App stack recommendation

CLIENT

Flutter

Android + iOS + web from one codebase; good fit for field + dashboard MVP.

MAPS

MapLibre + OSM

Avoid hard dependency on a proprietary map vendor; support custom tiles later.

REALTIME

WebSocket / SSE

Push issue/status updates to operator screens without polling.

AUTH

Keycloak OIDC/JWT

Self-hosted role-based access for admin, department, supervisor, field team.

5. Recommended Tech Stack

Opinionated stack for a strong MVP that can evolve into production.

Layer	Recommended choice	Why
Vehicle compute	Jetson Orin Nano Super; Orin NX if multi-camera/heavier models	Strong edge Vision AI ecosystem; compact; TensorRT acceleration.
Camera/GPS	1080p dashcam/CSI or USB camera + GNSS module	Enough visual detail for road defects; synchronized geolocation.
Video pipeline	GStreamer + OpenCV	Capture, decode, sampling, tracking, clip extraction.
Vision model	Fine-tuned YOLO26n/s	Real-time detector; choose nano first, small if accuracy requires.
Inference	TensorRT FP16 -> INT8 after calibration	Low-latency optimized inference on NVIDIA edge hardware.
Offline queue	SQLite + NVMe/filesystem	Durable store-and-forward when cellular connectivity drops.
API/backend	Python 3.12+ + FastAPI	Fast async-friendly ingestion and clean typed APIs.
Primary DB	PostgreSQL + PostGIS	Issue lifecycle plus spatial radius/nearest-neighbor queries.
Cache/jobs	Redis + Celery/RQ	Rate limits, short-lived state, background clustering/media tasks.
Media	MinIO (self-hosted) or S3	Object storage for images and short evidence clips.
App	Flutter	Single product codebase across Android/iOS/web.
Maps	MapLibre + OpenStreetMap/vector tiles	Flexible map UI and self-hosting path.
Identity	Keycloak	OIDC/JWT, RBAC, self-hosted government-friendly auth.
Observability	Prometheus + Grafana + structured logs	Device/API health, latency, queue depth and fleet visibility.
Deployment	Docker Compose for MVP; Kubernetes only when scale demands	Avoid premature orchestration complexity.

6. Build Order, Reliability & Security

Build the minimum closed loop first; optimize accuracy and scale after field data exists.

PHASE 1 - DEMO

- 1 vehicle
 - 3 classes
 - local detection
 - API upload
 - live map
 - issue detail
- Goal: prove autonomous detection end-to-end.**

PHASE 2 - MVP

- offline queue
 - PostGIS clustering
 - assignment
 - SLA
 - resolution evidence
 - re-verification
- Goal: prove closed-loop operations.**

PHASE 3 - PILOT

- multi-vehicle fleet
 - model retraining
 - RBAC
 - audit logs
 - analytics
 - observability
- Goal: validate in real roads and departments.**

PHASE 4 - SCALE

- HA backend
 - multi-city partitioning
 - Kubernetes if needed
 - model/version governance
 - retention policies
- Goal: operational resilience and governance.**

Non-negotiables



Idempotency

Each event upload carries a stable event_id so retries never create duplicate records.



Encryption

TLS in transit; encrypt device storage and server object storage; rotate credentials.



RBAC + audit

Every assignment/status change should record who, what, and when.



Model governance

Store model_version and confidence with every event; retraining must be measurable.

Design basis and current official references

- Your supplied flowchart: continuous dashcam -> rolling buffer -> ML detection -> evidence/metadata -> offline queue/upload -> backend deduplication -> assignment -> resolution -> independent vehicle verification.
- NVIDIA Jetson Orin Nano Super official documentation (updated Jun 2026): up to 67 INT8 TOPS, 7-25 W, 8 GB LPDDR5.
- Ultralytics official YOLO26 TensorRT integration documentation; NVIDIA TensorRT official Jetson documentation.
- PostGIS official manual for geospatial storage/indexing and index-aware spatial queries; FastAPI official async documentation; Flutter official web/platform documentation.